Calories Tracker

Machine Learning for Calories-Burned Estimation and Food Nutrition Tracking

1. Introduction

Most of us have looked at a fitness app and wondered how it came up with the calorie number it showed. The truth is that calories burned in a workout depend on a lot of things: how long we exercise, our height and weight, how fast the heart is beating, and how hard the workout actually is. A fixed rule of thumb gives a rough idea, but it does not treat every person or every workout fairly.

In this project, Calories Tracker, we tried a data-driven approach instead. We trained several regression models to predict calories burned from activity and body-related features, and then put the best one inside a simple interface that anyone can use. Alongside it, we added a food database for looking up calories in what a person eats, plus a BMI calculator and a daily energy estimate based on standard formulas. These parts sit in the same app, but they do different jobs. The machine-learning model estimates energy *burned*, while the food calculator estimates energy *consumed* from nutrition data.

1.1 Problem statement

Generalised calorie estimates often do not match the way a person actually works out. We wanted to see whether structured workout data could predict calories burned reasonably well, and how such a prediction could be shown in a friendly interface. We also ran into a second, more practical problem early on: data quality. Missing values, duplicate rows, mixed data types and unsuitable columns can damage a model before training has even started.

1.2 Objectives

- Clean and check both the activity/BMI dataset and the food nutrition dataset.
- Set up a training and testing workflow that others can repeat.
- Compare at least four regression models using MAE, RMSE and R^2.
- Pick the final model based on its performance on held-out test data.
- Build a UI for calories-burned estimation and food-calorie lookup.
- Include BMI and daily energy estimates, with clear notes on their limits.

1.3 Scope and responsible use

This is an educational estimator. It is not a medical device, and it cannot replace laboratory measurement. The results we report describe how the model did on the particular train/test split used

in our notebook. They do not promise the same accuracy for every person, every sensor or every kind of activity. Users should treat the output as an estimate, not an exact reading.

2. Literature Review

Research on estimating energy expenditure has moved steadily toward combining physiological signals, movement sensors and machine learning. We looked at studies that ask a question close to ours: can measurable activity and personal characteristics tell us how much energy someone uses? These studies use different datasets and evaluation set-ups from ours, so their numbers should not be lined up against ours as if everyone were tested on the same benchmark.

2.1 Jeran et al. (2022)

Jeran and colleagues built models to predict activity-related energy expenditure in freeliving adults, using accelerometer-based activity data together with extra participant information. One thing their work makes clear is that movement data alone does not explain everything. Body composition and lifestyle factors add useful information. This matters for us because it reminds us to choose features that describe both the activity and the person. Their setting was free-living life measured with accelerometers, which is quite different from the structured workout table we used. [1]

2.2 Burned Calories Prediction using Supervised Machine Learning (2022)

This IEEE conference paper compares several supervised regression methods for predicting calories burned and reports Random Forest as the best in its own experiment. That supports the idea of including tree-based ensembles in our comparison. At the same time, it does not mean Random Forest wins everywhere. Rankings can change with how features are defined, how data is collected and how the model is validated. So rather than assume, we compared Random Forest against several other models on our own held-out test set. [2]

2.3 Dynamic personalised energy expenditure with LSTM models (2023)

A 2023 study tried time-series methods, including LSTM, CNN-LSTM and an ensemble that mixes sequence modelling with gradient boosting. The authors stressed that energy expenditure changes over time, and that a personalised model can pick up patterns a general one misses. We see this as a good direction for future work. Our model works on one row of structured features per workout, not on continuous sensor sequences, so it should not be described as a dynamic time-series predictor. [3]

2.4 What these studies tell us

Taken together, the papers keep repeating one message: the quality of the features, the context of the measurement and the way a model is validated matter as much as the algorithm. They also warn

against trusting a single model or a single metric. We tried to follow that advice by comparing several regressors on the same test split and reporting error in kcal as well as R^2.

2.5 Recent work on wearable sensing and energy expenditure

Lendt et al. (2024) proposed a composite model that first classifies the type of activity and then applies a stride-specific energy estimate, using thigh-worn accelerometers. Their results suggest that telling walking, running and cycling apart is useful, because the link between movement and energy cost is different for each. Our project does not classify activity type yet, but this looks like a sensible extension. [4]

The 2024 CalAid paper describes a mobile fitness app that uses smartphone motion sensors, activity recognition and metabolic-equivalent (MET) information to estimate energy expenditure. What stands out is that it is not only a model but a complete app that connects sensing to a user interface. That is close to our own goal of making a model usable through a simple screen, although we rely on values typed in by the user rather than a live phone-sensor stream. [5]

Brønd (2024) studied LSTM recurrent neural networks for predicting energy expenditure from accelerometer data, which supports sequence-based modelling when the observations have a time structure. For us, that would need proper time-series input and validation that keeps each participant or session separate. Simply running an LSTM on ordinary table rows would not make the results more reliable. [6]

2.6 Comparison of the literature

Study	Year	Approach / data	Relevance to Calories Tracker
Jeran et al. [1]	2022	Accelerometry + participant factors	Use meaningful activity and personal features
IEEE supervised ML [2]	2022	Regression; Random Forest reported best	Include tree ensembles in the comparison
Study	Year	Approach / data	Relevance to Calories Tracker

Dynamic personalised EE [3]	2023	LSTM / CNN-LSTM / ensemble	Time-based and personalised models may help
Lendt et al. [4]	2024	Activity recognition + stridespecific model	Activity type can change the estimate
CalAid [5]	2024	Mobile app, phone sensors, activity recognition	Link the estimator to a usable interface
Brønd [6]	2024	LSTM on accelerometry	Time-series models are a future option

2.7 Research gap and where this project fits

Most of the work we reviewed depends on wearable sensors, activity sequences or carefully measured laboratory and free-living outcomes. Our project is simpler. It shows a complete workflow on tabular data: cleaning, feature preparation, model comparison, saved model files and an interactive calorie tracker. Its contribution is an accessible implementation and an honest benchmark on the data we had. We are not claiming to beat research-grade wearable systems. One clear limitation is that our notebook has not shown that the model generalises to new people or new devices.

3. Methodology

We followed a fairly standard machine-learning pipeline. The notebook loads two CSV files, checks their structure, cleans each one separately, defines the target, prepares the predictors and splits the activity data into training and test sets. It then fits several regression models, compares their test metrics and saves the chosen model along with supporting files.

3.1 Data sources and target

The activity/BMI dataset started with 15,020 rows and 10 columns. After removing exact duplicates and validating the data, 15,000 rows remained, and 14,992 of them had a known target value that we could use for training. The target is Calories_Burned_kcal. The food nutrition dataset started with 2,035 rows and 8 columns and came down to 2,025 rows after duplicates were removed and the data was cleaned.

3.2 Input features

The model uses eight predictors: Gender, Age_years, Height_cm, Weight_kg, Duration_min, Heart_Rate_bpm, Body_Temp_C and BMI. Seven are numeric and Gender is categorical. This is a reasonable set for estimating energy use because it covers workout length, the body's

response (heart rate and temperature) and basic body measurements. What it lacks is a directly recorded activity label or a continuous accelerometer signal.

3.3 Training and evaluation design

We split the data 80/20 into training and test sets with random_state = 42, which gave 11,993 training rows and 2,999 test rows. Preprocessing and the model are combined in one pipeline that handles missing values and encodes the different feature types. Keeping everything in a single fitted pipeline means the same transformations are applied at training time and at prediction time, which avoids a lot of small mistakes.

3.4 Model selection

We compared a mean-value Dummy Regressor (as a baseline) with Linear Regression, Ridge Regression, Decision Tree, Random Forest, Extra Trees and Gradient Boosting. The final model was chosen by the lowest test RMSE in the saved comparison output, and in the submitted run that was Gradient Boosting. For a more rigorous study, hyperparameter tuning should be done only inside the training data using cross-validation, and the test set should be kept aside for one final, unbiased estimate.

3.5 Evaluation metrics

Metric	Meaning	How to read it
MAE (kcal)	Average absolute gap between actual and predicted calories	Lower is better, and it is easy to understand in kcal
RMSE (kcal)	Square root of the average squared error	Lower is better, and large errors are punished more
R^2	Share of the target's variation explained, compared with a mean baseline	Closer to 1 is better, and it can go negative

4. Data Cleaning and Preprocessing

A large part of this project went into preparing the data, because a model cannot learn reliable patterns from messy inputs. We applied cleaning steps to both datasets before any modelling began.

4.1 Initial audit and duplicate removal

The audit looks at dataset size, data types, missing values and exact duplicate rows. We found 20 duplicate rows in the activity/BMI data and 5 in the food data. Removing them left 15,000 activity

records and 2,025 food records. This stops identical records from being counted more than once, although removing exact duplicates will not catch every data-quality problem.

4.2 Type conversion and validation

Column names were stripped of extra whitespace. Columns that should be numeric were converted carefully so that bad text does not quietly stay in the data as if it were a number. We checked the target before modelling and left out rows with no known target value. The predictors were also kept separate from the target, so the answer could not accidentally slip in as an input.

4.3 Feature processing

Numeric and categorical columns were handled separately. Numeric features go through an imputation step, while Gender is treated as a category and encoded. We preferred this to giving gender arbitrary numbers, because the model should not assume there is a numeric distance between labels. All of these steps live inside the notebook's pipeline and are saved together with the estimator.

4.4 Food database cleaning

The food table was cleaned on its own and has 2,025 items after preprocessing. The lookup uses the foodname field and the calories-per-100-gram field, and the calories for a logged amount are worked out proportionally. A 150 g serving, for example, uses 1.5 times the listed per-100 g value. This assumes that the database value is a fair representation of the selected food and serving.

4.5 Remaining data-quality risks

We removed exact duplicates and fixed types, but we do not claim that every problem is gone. Real datasets can still hide impossible body or heart-rate values, unit mix-ups, wrongly labelled records and odd outliers. Before wider use, values should be checked against domain rules, units should be documented and missing-data patterns should be reviewed. There is another point worth noting. If several records come from the same person, a random row split can leave that person's patterns in both the training and test sets. A person-level split would give a better picture of how the model behaves for users it has never seen.

4.6 Reproducibility and saved outputs

The exported bundle contains the selected model, the cleaned activity/BMI data, the cleaned food data, the model comparison table and a metadata JSON file. With these files, the experiment is easy to inspect, and the app can reuse the fitted model without retraining it every time.

5. Results and Discussion

5.1 Dataset summary

Item	Reported value
Activity/BMI data before cleaning	15,020 rows × 10 columns
Activity/BMI data after cleaning	15,000 rows × 10 columns
Rows with known target	14,992
Food nutrition data before cleaning	2,035 rows × 8 columns
Food nutrition data after cleaning	2,025 rows × 8 columns
Train / test rows	11,993 / 2,999
Selected target	Calories_Burned_kcal

5.2 Selected model performance

Gradient Boosting did best in our results, with MAE = 20.453 kcal, RMSE = 25.510 kcal and R^2 = 0.9711. Put simply, the model's predictions on the test set are off by about 20.5 kcal on average, and the high R^2 tells us it explains most of the variation in the held-out calorie values on this split. RMSE is higher than MAE, which suggests that a few larger misses are pulling the squared-error measure up.

Metric	Result	What it tells us
MAE	20.453 kcal	Typical size of the error
Metric	Result	What it tells us
RMSE	25.510 kcal	Sensitive to bigger misses
R^2	0.9711	Strong fit on this test split

5.3 Comparison with the simple baseline

The Dummy Regressor, which just predicts the mean, had an MAE of 129.412 kcal, an RMSE of 150.163 kcal and an R^2 slightly below zero. Gradient Boosting is far better than this on the same test split. That is encouraging, but a high R^2 on its own does not prove clinical accuracy or good performance on new populations. How the data was generated, how it was sampled and how similar the training and test records are can all make the score look better than it would be in real use.

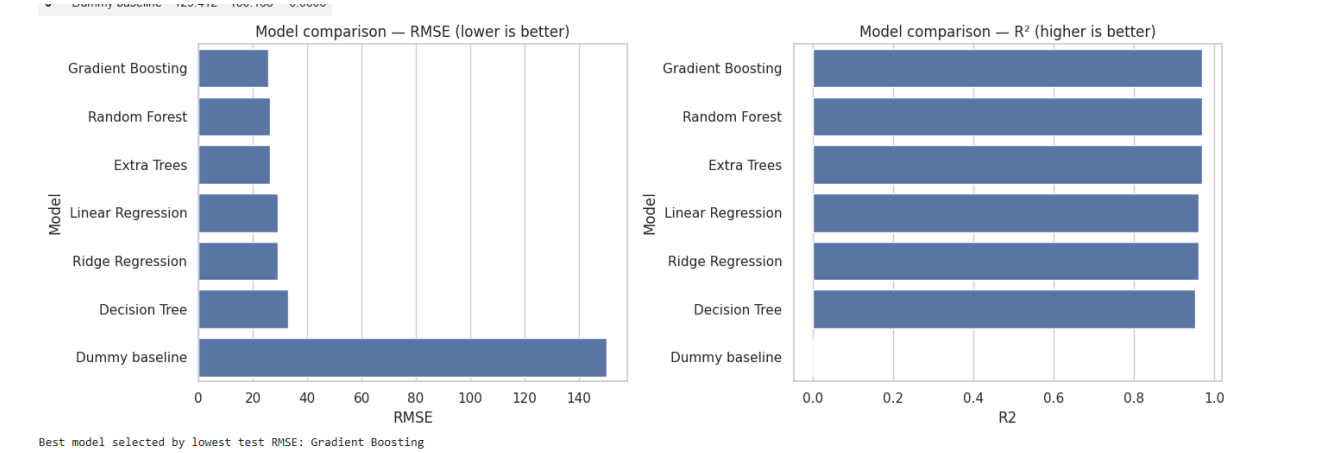
5.4 Interpretation

The results support our main goal: a trained regression model can give useful estimates from the tabular features we had. Tree-based ensembles did especially well, and Linear and Ridge Regression also performed strongly. A good next step would be an error analysis by workout duration, heart-rate

range, age group and body measurements, to find where the predictions are weakest. If the dataset has participant IDs, the evaluation should also be repeated with a split that keeps participants apart.

6. Benchmark Model Comparison

The table below uses the saved test-set metrics from our submitted artifact bundle. Every model was evaluated on the same target and the same split. MAE and RMSE are in kcal, so lower is better. R^2 has no unit, and higher is better.



Model	MAE (kcal)	RMSE (kcal)	R^2
Gradient Boosting	20.453	25.510	0.9711
Random Forest	20.934	26.229	0.9695
Extra Trees	21.125	26.389	0.9691
Linear Regression	23.064	29.225	0.9621
Ridge Regression	23.064	29.225	0.9621
Decision Tree	26.296	32.958	0.9518
Dummy baseline	129.412	150.163	-0.0003

6.1 Model-by-model discussion

Gradient Boosting ranks first by RMSE, with Random Forest and Extra Trees very close behind. Linear Regression and Ridge Regression give almost identical scores, which suggests that Ridge's regularisation made very little difference in this run. The Decision Tree does worse than the ensembles, which fits the idea that combining many trees makes predictions more stable. These are observations about our dataset and our split, not general rankings of the algorithms.

7. User Interface Development and Deployment

The screen recording shows a dark-themed Calories Tracker running locally in a browser. The page is built around one direct task: enter the user and workout details, calculate BMI and ask for a calories-burned estimate. Inputs are grouped together and the result is clearly highlighted, so the main output is hard to miss.

7.1 Main interface behaviour

The notebook also includes a food logging and nutrition calculator, a formula-based daily calorie-needs estimate and a BMI calculator. To keep things clear: the calorie-burn model predicts energy spent, the food database estimates energy eaten from the chosen food and quantity, the daily energy estimate comes from a formula and not from machine learning, and BMI is a screening value worked out from height and weight.

7.2 Deployment considerations

The notebook exports a joblib model file and the cleaned data files. A real deployment should load the fitted preprocessing-and-model pipeline, validate what users type in, deal with missing or out-of-range values and show units the same way every time. Our recording only demonstrates the app running locally. It does not show that a public cloud deployment is live. Before any public release, the project should be tested in a clean environment with pinned dependencies, the file paths should be checked, and sensitive user data should be kept out of logs.

7.3 Usability, privacy and limitations

A clear interface should label its inputs, say plainly that the result is an estimate and avoid presenting calorie numbers as medically exact. Body measurements and activity data are personal, so a production version should explain how data is stored and should not keep user inputs unless it needs to. Possible improvements include a layout that works well on mobile, prediction history with the user's consent, warnings for unusual input values, uncertainty estimates and an explanation of which features influenced the result most.

7.4 Future scope

In the future we could add participant-level validation, external testing, activity-type information, wearable sensor streams, time-series models and calibration for individual users. The nutrition module could also grow, with serving-size guidance and clearer sources for the food data. Each of these should be tested separately, so that a nicer interface is not mistaken for a more accurate model.

8. Conclusion

Calories Tracker brings together data cleaning, regression benchmarking, model export and a user-facing calorie estimation tool. On our test split, Gradient Boosting gave the lowest RMSE (25.510 kcal), with an MAE of 20.453 kcal and an R^2 of 0.9711. Random Forest and Extra Trees were close behind, while the simple mean baseline was much worse. The results show why it pays to compare several models instead of picking one by intuition.

The project also keeps three easily confused tasks apart. Calories burned come from the machine-learning model, food calories come from the nutrition database, and daily energy needs and BMI come from formulas. We think this separation makes the app more transparent for users. Overall, the current version is a promising educational prototype. It still needs external validation, evaluation with participants kept separate, stronger input checks and careful deployment before anyone relies on it beyond the dataset we used.

9. References

[1] S. Jeran et al., "Prediction of activity-related energy expenditure under free-livingconditions using accelerometer-derived physical activity," Scientific Reports, vol. 12, article 16578, 2022. doi:10.1038/s41598-022-20639-0. https://www.nature.com/articles/s41598-022-20639-0

[2] "Burned Calories Prediction using Supervised Machine Learning: RegressionAlgorithm," in 2022 Second International Conference on Power, Control and Computing Technologies (ICPC2T), 2022. doi:10.1109/ICPC2T53885.2022.9776710. https://ieeexplore.ieee.org/document/9776710

[3] "Dynamic personalized human body energy expenditure: Prediction using time seriesforecasting LSTM models," Biomedical Signal Processing and Control, 2023. https://www.sciencedirect.com/science/article/pii/S1746809423008145

[4] Scikit-learn developers, "scikit-learn: Machine Learning in Python," softwaredocumentation. https://scikit-learn.org/

[5] Streamlit, "Streamlit documentation," software documentation. https://docs.streamlit.io/